



Seminar Cuối Kì

Môn học: Nhập môn Dữ liệu lớn

A triple of Vectorstore Qdrant, Weaviate, and Milvus

Chủ đề 22 – NO1: Vector Store cho RAG workflows

Sinh viên thực hiện:

23120099 – Lê Xuân Trí
23120185 – Nguyễn Hồ Anh Tuấn
23120166 – Trần Hữu Kim Thành
23120165 – Trần Lê Trung Trực

Nhóm: Group 4T

Giảng viên hướng dẫn:

Thầy Trần Huy Bân

- 1 Giới thiệu
- 2 Qdrant
- 3 Weaviate
- 4 Milvus
- 5 So sánh
- 6 Đánh giá thực nghiệm
- 7 Kết luận

Mục tiêu thuyết trình

Phân tích **Qdrant**, **Weaviate** và **Milvus** như ba lựa chọn kỹ thuật tiêu biểu cho tầng retrieval trong hệ thống RAG.

Mạch trình bày

- Vai trò của Vector Database trong kiến trúc RAG.
- Kiến trúc và kỹ thuật đặc trưng của từng công cụ.
- So sánh, benchmark và tiêu chí lựa chọn cho production.



Vì sao cần Vector Database?

LLM thiếu dữ liệu mới

Không tự biết tài liệu nội bộ, chính sách mới hoặc dữ liệu sau thời điểm huấn luyện.

Cần evidence

RAG phải lấy đúng đoạn tài liệu liên quan trước khi sinh câu trả lời.

Vector DB là tăng retrieval

Lưu embedding, tìm Top-K chunks gần nghĩa và đưa context vào prompt.

SQL/NoSQL mạnh ở exact match; Vector Database mạnh ở tìm kiếm theo ngữ nghĩa.



Kiến trúc pipeline RAG

Indexing

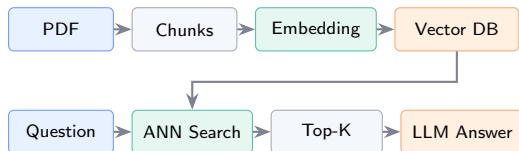
PDF → chunks →
embeddings → Vector DB

Answering

Question → Top-K context
→ LLM answer

Benchmark đo gì?

Latency, Recall@K, MRR,
tradeoff và DX.



Hình: Index trước, retrieval khi người dùng hỏi.



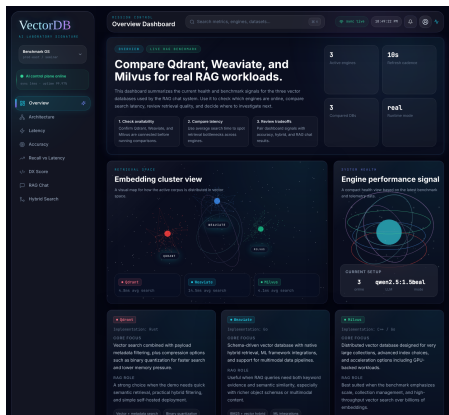
Hệ thống demo: từ benchmark đến dashboard

Không chỉ là slide lý thuyết

- React + Vite + Three.js dashboard.
- FastAPI điều phối ingest, chat và benchmark.
- Ollama local: `omic-embed-text`, `qwen2.5:1.5b`.

Fair HNSW config

Cosine, $M = 16$,
 $ef_construction = 128$,
 $ef_search = 64$.



Ba hệ quản trị Vector Database tiêu biểu

Qdrant

Rust, 2021

Filtered retrieval,
payload index,
quantization.

Weaviate

Go, 2019

Hybrid Search,
schema-driven,
ML/RAG-native.

Milvus

Go + C++, 2019

Distributed scale,
Knowhere, HA.

Ba công cụ giải cùng bài toán vector search, nhưng tối ưu cho ba ràng buộc khác nhau.



Độ phổ biến và cộng đồng GitHub

Qdrant

~31.4K

Tăng nhanh nhờ làn sóng RAG và AI agents.

Weaviate

~16.2K

Cộng đồng ổn định, mạnh về DX và Hybrid Search.

Milvus

~44.4K

Lâu đời, enterprise lớn, LF AI & Data.

GitHub stars cho thấy tín hiệu cộng đồng, không thay thế benchmark latency hay Recall@K.



Mã nguồn mở và mô hình triển khai

Bảng: License và các lựa chọn triển khai chính.

	Qdrant	Weaviate	Milvus
License	Apache 2.0	BSD-3	Apache 2.0
Self-hosted	Docker/K8s	Docker/K8s/Helm	Docker/K8s/Operator
Managed Cloud	Qdrant Cloud	Weaviate Cloud	Zilliz Cloud
Pricing	OSS miễn phí; cloud trả phí khi scale	OSS miễn phí; cloud trả phí sau sandbox	OSS miễn phí; Zilliz trả phí theo usage
Governance	Company-led	Company-led	LF AI & Data

Lưu ý khi đọc bảng

Cả ba đều có thể triển khai local/self-host không mất phí license. Khác biệt thực tế nằm ở chi phí hạ tầng, độ phức tạp vận hành, SLA cloud và mô hình governance.



Free tier cloud và giới hạn thử nghiệm

Qdrant Cloud

1 node, khoảng 1GB RAM và 4GB storage.

Weaviate Cloud

Sandbox 14 ngày, phù hợp thử nghiệm ngắn.

Zilliz Cloud

Free cluster cho khoảng 1M vectors hoặc 5GB storage.

Free tier giúp prototype; production cost chủ yếu nằm ở RAM, storage, compute, backup và SLA.



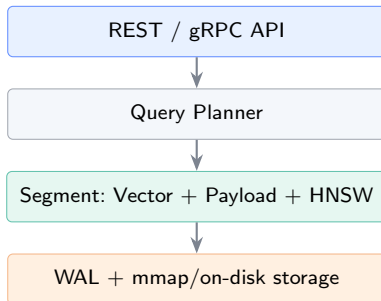
Qdrant: kiến trúc tối giản cho filtered retrieval

Đặc điểm triển khai

- Single binary, ít dependency.
- Segment giữ vector, payload và HNSW.
- WAL bảo vệ write path.

Điểm cần nhớ

Qdrant tối ưu cho RAG có metadata filter như tenant, ACL, timestamp.



Ít service hơn → vận hành nhẹ hơn

Hình: Qdrant gom query path quanh segment và payload.



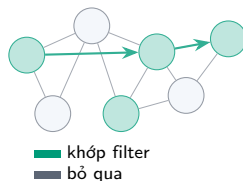
Qdrant và bài toán truy hồi có filter

Ý tưởng chính

- HNSW là ANN index dạng đồ thị.
- Filter không chỉ được xử lý ở bước hậu xử lý.
- Qdrant đưa **payload filter** vào quá trình duyệt graph.
- Khi filter có độ chọn lọc cao, planner có thể đổi sang chiến lược phù hợp hơn.

Adaptive behavior

Filter phổ biến → Filterable HNSW.
Filter rất hẹp → payload index + brute-force.



Hình: Filter được đưa vào quá trình traversal.



Ba chiến lược kết hợp vector search và metadata

Pre-filter

Lọc metadata trước rồi search trên tập con.

Rủi ro: graph bị thưa.

Post-filter

Search vector trước rồi loại kết quả sai filter.

Rủi ro: thiếu Top-K hợp lệ.

In-graph

Kiểm tra filter ngay khi duyệt HNSW.

Hợp: filter-heavy RAG.

Trong production RAG, filter ảnh hưởng trực tiếp đến Recall@K, latency và tenant isolation.



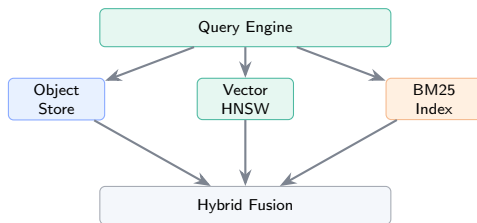
Weaviate: schema mô-đun cho ứng dụng RAG

Ba chỉ mục cùng tồn tại

- Object Store: dữ liệu và metadata.
- HNSW: semantic retrieval.
- Inverted Index: BM25 và filter.

ML-native

Hỗ trợ workflow gần ML/RAG app, multimodal và tích hợp với hệ sinh thái PyTorch/TensorFlow.



Hình: Keyword và semantic trong cùng query engine.



Hybrid Search: kết hợp keyword và semantic retrieval

BM25F

Mạnh với keyword chính xác, tên riêng, mã lỗi, thuật ngữ hiếm.

Vector

Mạnh với paraphrase, đồng nghĩa, truy vấn theo ý nghĩa.

α

0.0 = keyword

0.5 = cân bằng

1.0 = vector

Weaviate phù hợp khi tài liệu vừa cần exact terms, vừa cần semantic similarity.



RRF: hợp nhất hai bảng xếp hạng

Reciprocal Rank Fusion

$$\text{score}(d) = \sum_{i=1}^m \frac{1}{k + \text{rank}_i(d)}$$

Bảng: Ví dụ fusion từ BM25 và vector ranking.

Document	BM25 rank	Vector rank	Ý nghĩa
d_1	1	5	keyword rất mạnh
d_2	8	1	semantic rất mạnh
d_3	20	18	yếu ở cả hai

Rank position thay cho raw score

BM25 score và cosine similarity khác thang đo; RRF dùng vị trí xếp hạng nên ổn định hơn khi hợp nhất hai nguồn tín hiệu.



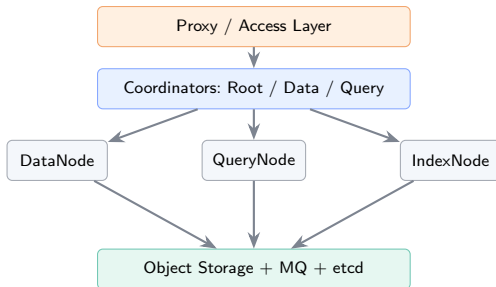
Milvus: Kiến trúc phân tán shared-storage

Bốn lớp kiến trúc

- Proxy nhận request.
- Coordinator điều phối metadata.
- Worker xử lý ingest/search/index.
- Storage tách khỏi compute.

Tradeoff

Scale tốt hơn, nhưng vận hành phức tạp hơn Qdrant/Weaviate.



Hình: Milvus tách compute khỏi storage.



Scale phân tán & Knowhere Engine

Scale đúng điểm nghẽn

Search chậm → QueryNode

Ingest chậm → DataNode

Build index chậm → IndexNode

Storage tăng → MinIO/S3

Knowhere

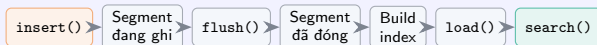
Abstraction layer cho Faiss, HNSWlib, ScaNN, DiskANN; hỗ trợ SIMD path và GPU acceleration khi có CUDA.

Qdrant/Weaviate đơn giản hơn; Milvus đáng giá khi corpus và concurrency rất lớn.



Vòng đời dữ liệu & bất vận hành

Vòng đời chuẩn



Growing vs Sealed Segment

Growing Segment có thể searchable bằng brute-force; Sealed Segment được persist ra object storage và có full index. **Lưu ý:** gọi `flush()` quá thường xuyên với batch nhỏ tạo nhiều sealed segments, gây I/O amplification và giảm throughput.



Milvus HA & khả năng chịu lỗi

Các cơ chế HA

- etcd quorum cho metadata consistency.
- Coordinator failover qua service discovery.
- QueryNode replica cho serving availability.
- Message Queue lưu ingestion stream và WAL-like durability.

Tính năng nâng cao

DiskANN cho dataset vượt RAM, Time Travel theo timestamp, CDC cho replication, GPU Index cho build/search acceleration.

Chi phí production: HA cần monitoring, backup policy, resource planning và version compatibility.



Ma trận so sánh kiến trúc

Qdrant

Filtered performance

Single binary, payload index, filterable HNSW, triển khai nhẹ.

Weaviate

Hybrid RAG ergonomics

Object + vector + inverted index trong cùng schema-driven engine.

Milvus

Distributed scale

Proxy, coordinators, workers, object storage, HA và Knowhere.

Câu hỏi đúng không phải "DB nào tốt nhất", mà là "ràng buộc nào quan trọng nhất".



Chiến lược Vector Quantization

Kỹ thuật nén

- Scalar: float32 $\rightarrow$ uint8.
- Product: chia vector thành sub-vectors.
- Binary: vector $\rightarrow$ 1-bit đại diện.

Con số đáng nhớ

Qdrant nhấn mạnh Binary Quantization có thể tăng tốc comparison lên đến **40x** với trade-off Recall cần đo lại.

Kỹ thuật	Qdrant	Weaviate	Milvus
Scalar Quantization	✓	×	✓
Product Quantization	✓	✓	✓
Binary Quantization	✓	✓	×
Asymmetric / rescoring control	✓	✓	✓
Disk-oriented vector search	✓	×	✓



Giao thức benchmark công bằng

Biến được kiểm soát

- Cùng corpus, chunk size, overlap và preprocessing.
- Cùng embedding model và vector dimension.
- Cùng query set, Top-K và random seed.
- Cùng hardware envelope, Docker limits và warm-up policy.

Hardware: AMD Ryzen 7 5800H, 16 logical CPUs, 14GiB RAM, NVMe SSD 512GB, Arch Linux.

Nguyên tắc diễn giải: mỗi database có lifecycle và planner riêng; benchmark phải ghi rõ config. Không tính bước cài đặt, đăng ký tài khoản hay thao tác cloud console.

Metric đo lường

Latency p50/p95, RAM footprint, insert/build cost, Recall@K, MRR và DX score.



Telemetry tài nguyên: latency & RAM



Hình: Dashboard benchmark latency/RAM tập trung vào vùng biểu đồ chính.

Đọc biểu đồ

- Weaviate có p50 thấp nhất trong search path.
- Qdrant ổn định hơn ở p95 và footprint thấp.
- Milvus search tốt sau khi load, nhưng lifecycle nặng hơn.

Lưu ý: latency search không đại diện cho toàn bộ chi phí vận hành.



Độ chính xác truy hồi: Recall@K & MRR

Định nghĩa metric

$$\text{Recall@K} = \frac{|R_K|}{|R|}$$

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}$$

Rủi ro

Latency thấp nhưng truy hồi sai làm LLM thiếu evidence đáng tin cậy.

Nhận xét: Milvus có Recall@K/MRR cao nhất trong tập thử nghiệm; Qdrant và Weaviate cần tune thêm Top-K, hybrid alpha hoặc index params.



Hình: Recall@K và MRR từ dashboard benchmark.



Đánh đổi accuracy-speed & Pareto Frontier

Cách đọc Pareto

- Trục X: latency trung bình.
- Trục Y: Recall@K.
- Điểm tốt nằm cao và lệch trái.
- Chọn cấu hình theo SLA thay vì nhìn một metric đơn lẻ.

Ý nghĩa: Milvus thắng về recall trong snapshot; Qdrant/Weaviate phù hợp hơn khi ngân sách latency chặt.



Hình: Pareto Frontier thực nghiệm: accuracy so với p95 latency.



Ma trận Developer Experience

Bảng: Ma trận DX cho nhóm xây dựng RAG application.

Tiêu chí DX	Qdrant	Weaviate	Milvus
Thiết lập local	Rất dễ	Dễ	Khó hơn
Số lượng dependency	Thấp	Thấp-vừa	Cao
Độ dễ dùng Query API	Rõ ràng	Rất tốt cho RAG	Mạnh nhưng nghiêm ngặt
Tiến hóa schema	Linh hoạt payload	Auto/schema-driven	Strict FieldSchema
Gánh nặng observability	Thấp	Trung bình	Cao
Vận hành production	Nhẹ	Vừa	Nặng nhưng scale tốt

Kết luận DX

DX bao gồm setup, mental model, debugging, schema evolution, dependency graph và monitoring, không chỉ là số dòng code.



Ma trận quyết định cuối cùng

Khuyến nghị theo use case

- **Qdrant**: filter-heavy RAG, multi-tenant payload, footprint thấp, triển khai nhẹ.
- **Weaviate**: Hybrid Search, BM25F + semantic retrieval, RAG app cần DX tốt.
- **Milvus**: corpus rất lớn, High Availability, distributed production, GPU/DiskANN path.

Kết luận

Không có “best Vector Database” tuyệt đối; lựa chọn đúng phụ thuộc workload, SLA, corpus size và năng lực vận hành.



thank
you!